

รายงานแบบบันทึกการใช้งาน AI และการวิเคราะห์ระบบ

รายวิชา: โครงสร้างข้อมูลและอัลกอริทึม (Data Structures & Algorithms)

หัวข้อ: กลุ่มที่ 5 – Emergency Workflow: Event Stack และ State Machine

วันที่จัดทำ: 17 สิงหาคม 2026

1. แบบบันทึกการใช้งาน AI (AI Usage Log)

วันที่ / เวลา	งานที่ทำ	Prompt ที่ใช้สั่ง AI	ผลลัพธ์ที่ได้รับจาก AI	การตรวจสอบและการนำไปใช้
17/08/2026 10:15 น.	การออกแบบ State Machine	"ขอวิธีออกแบบ Transition Table ใน Java ให้เรียก Action ได้ O(1)"	แนะนำการใช้ HashMap ซ้อน HashMap เพื่อค้นหาคู่ (State, Action)	นำไปสร้าง TransitionTable.java กำหนดสแตตทั้ง 5 ขั้นตอน
17/08/2026 11:30 น.	กลไก Double Stack (Undo/Redo)	"เขียนฟังก์ชัน Undo/Redo ร่วมกับ eventStack และ redoStack โดยให้ล้าง Redo เมื่อมี Action ใหม่"	ให้โค้ดการย้าย Object ระหว่าง 2 Stack และการเรียก redoStack.clear()	นำไปปรับใช้ใน AlgorithmB.java พร้อมอัปเดต currentState

17/08/2026 14:00 น.	การสร้าง JUnit Test Cases	"ช่วยออกแบบ Test Cases ให้ครอบคลุม 10 สถานะ บังคับตาม โจทย์"	สร้างโครงสร้าง Test Case ครอบคลุม Normal Flow, Edge Cases และ Exception	นำไปใส่ใน <code>TestCases.java</code> และ รันสอบทานผ่านครบ 10/10 Cases
17/08/2026 16:45 น.	วิเคราะห์ Correctness Invariant	"ช่วยอธิบาย Correctness Invariant และตอบคำถามวิเคราะห์ ระบบ 5 ข้อ"	ให้คำอธิบายเชิง วิศวกรรมเรื่อง O(1) Lookup, Branching History และ Audit Log	ตรวจสอบความถูกต้องและเรียบเรียงสรุป เป็นภาษาตนเอง

2. ข้อกำหนดระบบและ Transition Table

ตารางเปลี่ยนสถานะ (Transition Table)

- **NEW** + **CALL_RECEIVED** $\rightarrow$ **RECEIVED**
- **RECEIVED** + **TEAM_ASSIGNED** $\rightarrow$ **ASSIGNED**
- **ASSIGNED** + **VEHICLE_DISPATCHED** $\rightarrow$ **DISPATCHED**
- **DISPATCHED** + **ARRIVED_AT_SCENE** $\rightarrow$ **ON_SCENE**
- **ON_SCENE** + **CASE_CLOSED** $\rightarrow$ **CLOSED**

กฎบังคับของระบบ (System Rules)

1. หลัง Undo 2 ครั้ง ต้องมี 2 Action อยู่ใน Redo Stack
2. Redo 1 ครั้ง คืบ Action ล่าสุดกลับสู่ Event Stack
3. หากเพิ่ม Action ใหม่หลัง Redo บางส่วน ต้องล้าง Redo Stack ทันที (`redoStack.clear()`)
4. **currentState** ต้องตรงกับ Action ล่าสุดใน Event Stack หลังทุกคำสั่ง
5. ห้ามเปลี่ยนสถานะข้ามขั้นตอน (Strict Transition)

3. บทวิเคราะห์และ Correctness Invariant

Correctness Invariant

ข้อกำหนด: หลังจบทุก Operation ค่า **currentState** ต้องตรงกับ Action ล่าสุดบนยอดสุดของ **eventStack** เสมอ และลำดับ Action ทั้งหมดใน **eventStack** ต้องเรียงเป็น Workflow ที่ถูกต้องตามกฎหมาย

เฉลยคำถามวิเคราะห์ 5 ข้อ

- 1. **Algorithm A ต้องค้นหา Event Stack กรณีใด:** ต้องค้นหา (Traverse) ย้อนหลังทุกครั้งที่มี Action ใหม่เข้ามา เพื่อตรวจสอบว่าลำดับเหตุการณ์ทั้งหมดถูกต้องหรือไม่ เนื่องจากไม่ได้เก็บ **currentState** ไว้ล่วงหน้า
- 2. **State Machine ช่วยให้ตรวจสอบเป็น $O(1)$ ได้อย่างไร:** ใช้ตารางเปลี่ยนสถานะ (Lookup Table) ในการตรวจสอบคู่ (**currentState**, **action**) เพื่อดึงสถานะถัดไปได้ทันทีโดยไม่ต้องวนลูปอ่านประวัติใน Stack
- 3. **การคืน currentState หลัง Undo หลายครั้ง:** ระบบจะย้าย Action ออกจาก **eventStack** ไปยัง **redoStack** แล้วนำ Action ที่อยู่บนยอดสุดของ **eventStack** ที่เหลือออกมาอัปเดตเป็น **currentState**
- 4. **ทำไมต้องสร้าง Redo Stack เมื่อเพิ่ม Action ใหม่:** เพราะการเพิ่ม Action ใหม่หลังจาก Undo คือการสร้างเส้นทางอนาคตใหม่ (Timeline Branching) Action เดิมที่เคย Undo ไว้จึงกลายเป็นอนาคตที่ไม่สอดคล้องกับปัจจุบันอีกต่อไป
- 5. **Event Log ในระบบจริงควรถูกลบเมื่อ Undo หรือไม่:** **ไม่ควรลบ** ในระบบจริงจะใช้รูปแบบ Append-Only Log โดยบันทึกเหตุการณ์ประเภท **UNDO** เพิ่มเข้าไป เพื่อบันทึกประวัติการทำงาน (Audit Trail) ให้สามารถตรวจสอบย้อนหลังได้

4. สรุปผลการทดสอบ (Test Cases Verification)

Case #	สถานการณ์ทดสอบ	พฤติกรรมที่คาดหวัง	ผลการทดสอบ
1	Workflow ถูกต้อง ตั้งแต่เริ่มจนจบ	เปลี่ยนสถานะตามลำดับจาก NEW จนถึง CLOSED	PASSED
2	เพิ่ม Action ผิดลำดับ	โยน Exception เตือน และปฏิเสธการเปลี่ยนสถานะ	PASSED
3	ปิด Case ก่อนถึงที่เกิดเหตุ	ปฏิเสธคำสั่ง CASE_CLOSED จากสแตต DISPATCHED	PASSED

4	Undo หนึ่งครั้ง	ย้าย 1 รายการไป Redo Stack และ ถอยสแตคกลับ 1 ชั้น	PASSED
5	Undo หลายครั้ง	ถอยสแตคกลับตาม Action ที่เลื่อนบน ยอด Event Stack	PASSED
6	Redo บางส่วน	ย้าย Action กลับมา Event Stack 1 รายการ และอัปเดตสแตคไปข้างหน้า	PASSED
7	เพิ่ม Action ใหม่หลัง Redo บางส่วน	ล้างรายการใน Redo Stack ทั้งหมด แล้วเริ่มบันทึก Action ใหม่	PASSED
8	Undo จน Stack ว่าง	Event Stack ว่างเปล่า และคืค่า currentState กลับเป็น NEW	PASSED
9	Redo เมื่อไม่มีรายการ	ระบบไม่เปลี่ยนแปลงโครงสร้าง หรือ แจ้งเตือน Redo Stack ว่าง	PASSED
10	เพิ่ม Action หลังปิด Case	ปฏิเสธคำสั่งใหม่ทั้งหมดเนื่องจากอยู่ ในสแตค CLOSED	PASSED